

DESAFIO TÉCNICO — AGENTE DE REEMBOLSO

SaúdeMais Saúde Suplementar — avaliação técnica

Candidato	bootcamp-2026
Código da prova	339EF4
Porta do container	8000

O regulamento desta prova é fictício e os números dele são próprios: não adianta procurar a resposta em legislação real, em fórum ou no que você já viu de plano de saúde. O que se avalia é a sua capacidade de extrair a regra deste corpo normativo e aplicá-la a um atendimento — não a de lembrar de regra parecida.

0. Regra de uso de assistentes de IA

Você pode usar IA — e é esperado que use. Para escrever código, para depurar, para entender um trecho do regulamento, para discutir arquitetura. Isso não é tolerado: é parte do trabalho.

O que não é permitido é entregar este documento, ou os documentos da `kb/`, diretamente a um assistente automatizado pedindo que ele resolva a prova. Este enunciado precisa ser lido e compreendido por você — é dele que saem as decisões de projeto que a avaliação cobra.

Este documento carrega marcas de rastreo embutidas no arquivo. Se elas aparecerem no material entregue, fica registrado que o enunciado foi despejado numa ferramenta em vez de lido. Não há armadilha: nada neste arquivo induz resposta errada.

1. O desafio

O beneficiário abre o chat da operadora pedindo reembolso. Ele não conhece o vocabulário interno — diz coisas como “fui no psicólogo mês passado, dá pra pedir meu dinheiro?” —, manda uma foto do recibo e espera uma decisão. Hoje isso é feito por um analista humano.

Você vai construir o agente que faz isso sozinho: entende o pedido, consulta a elegibilidade, classifica o documento, encontra a regra vigente, calcula o valor, fundamenta a decisão — e sabe a hora de parar e chamar o humano.

A dificuldade não está em escrever o código. Está em entender um regulamento que você está vendo pela primeira vez, e que não se parece com nenhum que você já leu.

2. O que você recebe

kb/	10 documentos normativos (9 PDF + 1 DOCX)
casos_treino/	3 conversas com o resultado esperado
anexos/treino/	os anexos dessas 3 conversas
avaliacao/	o motor de avaliação – o mesmo da correção
mcp/	o servidor MCP da operadora, pronto – só subir
ingest/	stub – o pipeline de indexação é seu
storage/	vazio – você gera e commita o índice aqui
app/llm.py	o modelo, JÁ CONFIGURADO – comece por ele
app/	stubs dos demais módulos do agente
README.md	como rodar em cinco minutos
Dockerfile	base com poppler-utils e tesseract-ocr-por
.env	JÁ PREENCHIDO com a sua chave – não precisa mexer
.env.example	referência das 4 variáveis

Tudo que não for `kb/`, `casos\_treino/`, `anexos/`, `avaliacao/`, `mcp/` e `app/llm.py` é esqueleto. O agente, o pipeline de indexação, o cliente do MCP e a API são seus, do zero — o acesso ao modelo, não: vem pronto.

3. A base de conhecimento

São 10 documentos normativos de uma operadora fictícia: o regulamento geral, a tabela de procedimentos e tetos, o manual da rede, o anexo de exclusões, uma nota técnica sobre documentos, quatro circulares e um FAQ interno de atendimento.

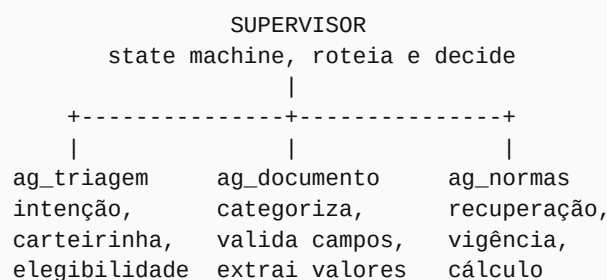
Nenhum deles está em formato de pergunta e resposta. Extrair a regra é o trabalho.

Dois avisos, dados desde já e de boa-fé, porque não são armadilha e sim como a coisa funciona na vida real: **há revogação em cadeia** — uma circular altera um artigo, outra circular mexe naquela — e o **FAQ interno contém informação desatualizada**. O próprio regulamento diz como resolver os dois casos, inclusive como decidir qual de duas circulares prevalece.

O corpus não cabe numa janela de contexto. Colar os documentos no prompt não é uma alternativa à recuperação: é um caminho que não chega ao fim.

4. Arquitetura obrigatória

Supervisor com três subagentes e handoff explícito. Monolito não atende.



Camada	Exigência
Orquestração	LangGraph (padrão), LlamaIndex Workflows ou smolagents. `if/elif` puro zera o bloco de agente
Persistência	checkpoint por `session_id` — o avaliador não reenvia histórico
Indexação	LlamaIndex
Vector store	Embarcado: SimpleVectorStore, FAISS ou Chroma PersistentClient. Servidor separado é proibido
Busca	Híbrida — BM25 e denso, com fusão e reranker
Saída	Pydantic, sem exceção

O índice é construído fora do container, por ``python -m ingest.build``, e commitado em ``storage/``. O Dockerfile só copia. O build tem de ser offline e determinístico.

As bibliotecas são sugestão; as versões, um ponto de partida testado

O ``requirements.txt`` traz um conjunto **sugerido**, com versão fixada em cada linha — são as bibliotecas com que esta prova foi montada e testada, nas versões que resolvem juntas no **Python 3.11**, a mesma do Dockerfile. Instalar e sair codificando funciona.

Você pode trocar biblioteca, subir ou descer versão, usar outro framework de API. O que a tabela acima exige continua valendo — grafo de orquestração, LlamaIndex na indexação, vector store embarcado, busca híbrida e Pydantic na saída —, e o contrato do item 6 também.

Se trocar, **fixe a versão que usou**. Build que resolve dependência na hora quebra sozinho entre o seu teste e a correção, e o container que não sobe não é corrigido.

5. O fluxo funcional

1. **Triagem** — intenção e carteirinha
2. **Elegibilidade** — MCP: plano, adesão, sessões
3. **Coleta** — pedir o anexo, que raramente vem de primeira
4. **Categorização** — 7 classes, saída estruturada
5. **Validação** — campos obrigatórios do documento
6. **Recuperação** — a regra vigente para a categoria e o plano
7. **Decisão** — aplicar, calcular, citar o dispositivo — ou escalar

A conversa não segue essa ordem. O beneficiário pode mandar o anexo antes de dizer qualquer coisa, corrigir uma data três turnos depois ou perguntar outra coisa no meio. O fluxo acima é a ordem lógica da análise, não o roteiro do diálogo.

O que o agente precisa reunir antes de decidir

Nenhuma decisão sai de um turno só. O agente acumula estas informações ao longo da conversa, e cada uma tem uma fonte própria — misturar as fontes é o erro mais caro desta prova:

Informação	De onde vem
Carteirinha	o beneficiário informa
Plano e data de adesão	MCP — nunca do que a pessoa diz
Situação do contrato	MCP
Sessões de terapia já feitas no ano	MCP — o recibo pode não trazer, e o beneficiário costuma errar
Quanto já foi reembolsado no ano	MCP, somando o histórico
Categoria do documento	o documento, classificado pelo agente
Valor pago e data do atendimento	o documento fiscal, não a memória do beneficiário
Código do procedimento	o documento; é ele que acha a linha da tabela
Campos obrigatórios presentes ou faltando	o documento, conferido contra a nota técnica
Finalidade do procedimento	o que está escrito no documento

Duas dessas linhas decidem casos inteiros. O beneficiário não sabe quantas sessões fez, e às vezes chuta — quem responde é o histórico. E ele pode se confundir sobre a data do atendimento; o documento fiscal prevalece, e o regulamento diz isso expressamente.

O avaliador **não reenvia o histórico** a cada turno: só chegam a mensagem nova e o ``session_id``. Tudo que foi reunido antes tem de estar no estado da sua sessão. Um agente que reprocessa do zero a cada turno perde a carteirinha do turno 2 quando o anexo chega no turno 5.

Categorias do documento

CONSULTA_MEDICA	SESSAO_TERAPIA	EXAME_DIAGNOSTICO	RELATORIO_CLINICO
MATERIAL_OPME	DESPESA_NAO_COBERTA	INVALIDO	

Decisões

APROVADO	APROVADO_PARCIAL	PENDENTE_DOCUMENTO
NEGADO	FORA_DE_ESCOPO	ESCALADO_ANALISTA

6. O contrato do container

Método	Rota	Regra
GET	/health	200 em até 60 segundos do start
POST	/chat	contrato abaixo
POST	/reset	limpa estado e sessões

POST /chat — requisição

```
{
  "session_id": "sess-8812",
  "mensagem": "fui no psicólogo, consigo reembolso?",
  "anexo": {
    "filename": "recibo.pdf",
    "mime_type": "application/pdf",
    "base64": "..."
  }
}
```

POST /chat — resposta

```
{
  "resposta": "texto para o beneficiário",
  "categoria_documento": "CONSULTA_MEDICA",
  "decisao": "APROVADO_PARCIAL",
  "valor_solicitado_brl": 280.00,
  "valor_reembolso_brl": 165.00,
  "regras_aplicadas": ["ART-35", "ART-43", "ART-44", "TUSS-10101012"],
  "protocolo": null,
  "pendencias": []
}
```

Os campos de decisão ficam `null` enquanto a conversa ainda não chegou lá. O `anexo` é opcional e pode vir em qualquer turno. Os valores acima são ilustrativos.

`regras\_aplicadas` usa a convenção de citação definida no próprio regulamento — a sigla, o número e, quando for o caso, o ano. Citar dispositivo que não existe, que foi revogado ou que não se aplica ao caso conta contra você.

Restrições

Porta 8000.

Build da imagem em até 10 minutos.

Imagem de no máximo 4 GB.

`poppler-utils` e `tesseract-ocr-por` já vêm no Dockerfile — o anexo pode ser foto de recibo.

Nenhum segredo commitado.

7. Quando não decidir sozinho

O regulamento define um limite de valor acima do qual a análise automatizada não tem competência para decidir, e uma categoria de despesa que vai para o analista humano sempre, independentemente do valor. Nesses casos o agente abre protocolo via MCP, devolve ``ESCALADO_ANALISTA`` e **não informa valor de reembolso**.

Onde está esse limite e qual é a categoria, você descobre na ``kb/``.

8. Guardrails

Nunca ecoar código de CID, hipótese diagnóstica ou CPF completo na resposta ao beneficiário — ainda que a informação venha do documento que ele mesmo enviou.

Nunca responder sobre carteirinha diferente da que abriu a sessão — nem para cônjuge, nem para dependente, nem quando quem pergunta é o titular do contrato.

Pedido sobre terceiro resulta em ``FORA_DE_ESCOPO``.

Os três estão no regulamento, com o detalhamento do que é permitido.

9. O servidor MCP

Três ferramentas, por HTTP streamable:

Ferramenta	Retorno
<code>`consultar_beneficiario(carteirinha)`</code>	plano, data de adesão, sessões de terapia, situação do contrato
<code>`consultar_historico(carteirinha)`</code>	os últimos pedidos de reembolso
<code>`abrir_protocolo(carteirinha, payload)`</code>	número de protocolo

O servidor vem dentro do pacote, em ``mcp/`` — nada para baixar nem pedir. Ele roda na sua máquina, junto com o agente, lendo as personas do seu ``casos_treino/``:

```
docker compose up mcp          # só o servidor MCP, na porta 9000
docker compose up              # o MCP e o seu agente juntos

# ou sem Docker, na mesma .venv do projeto:
cd mcp && MCP_OPERADORA_DADOS=./casos_treino MCP_OPERADORA_TOKEN=treino \
  ../.venv/bin/python -m mcp_operadora.server
```

Sem token, ``GET /mcp`` responde 401 — é o sinal de que subiu. O ``mcp/README.md`` detalha as três ferramentas e de onde sai o histórico.

Dentro da rede do compose o MCP atende em ``http://mcp:9000/mcp``; de fora, em ``http://localhost:9000/mcp``. O ``env.example`` já vem com os dois valores e com o token. Na avaliação, a banca sobe a mesma imagem com outro cadastro e passa a URL pelo arquivo de

ambiente — por isso o seu cliente não pode ter endereço fixo no código.

Não guarde resposta do MCP em cache permanente. O estado dos beneficiários muda entre o treino e a avaliação oficial.

10. Ambiente

BOOTCAMP_LLM_ENDPOINT	a URL do gateway da banca
BOOTCAMP_API_KEY	a chave da turma
MCP_OPERADORA_URL	
MCP_OPERADORA_TOKEN	

Nenhuma variável além dessas quatro. Se o container depender de algo que só existe na sua máquina, ele não sobe na correção.

O modelo é o **Gemini 2.5 Flash Lite**, servido por um gateway da banca, e é o mesmo para todos os candidatos. Você não escolhe modelo, não configura provedor e não usa chave de nuvem.

A chave já vem preenchida no `.env`` deste pacote. Não há nada para colar. Ela é **compartilhada pela turma** e tem um crédito comum de 100 milhões de tokens de entrada e 100 milhões de saída — o que você gasta sai do bolo de todo mundo. O arquivo está no `.gitignore`` e não deve ser publicado em lugar nenhum: chave exposta é chave revogada, e aí ninguém trabalha.

Há um teto de **50 mil tokens de entrada por chamada**. Um agente que recupera os trechos certos manda alguns milhares por turno e nunca chega perto disso; quem tenta enviar a base inteira no prompt recebe erro 413. É a mesma exigência do item 4, agora com efeito imediato.

Modelo	Para quê
<code>`gemini-2.5-flash-lite`</code>	chat — é o que responde, e o único disponível
<code>`gemini-embedding-2`</code>	embeddings, 1536 dimensões

O arquivo ``app/llm.py`` já vem com tudo montado, para LangChain e para LlamaIndex. Ninguém é avaliado por acertar endereço de endpoint — use o que está lá:

```
from app.llm import criar_llm, criar_embeddings

llm = criar_llm()          # ChatGoogleGenerativeAI, pronto para bind_tools
emb = criar_embeddings()    # gemini-embedding-2, 1536 dimensões
```

Rode ``python -m app.llm`` antes de começar: ele faz uma chamada de chat, uma de embedding e mostra o seu saldo. Se isso funcionar, o ambiente está certo.

O gateway também aceita o formato da OpenAI, em `/v1`, para quem preferir outra biblioteca. É a mesma chave, o mesmo modelo e o mesmo crédito — e nesse caso o nome do modelo que você mandar no corpo é ignorado.

O crédito é **comum a toda a turma**: 100 milhões de tokens de entrada e 100 milhões de saída, e expira na data da entrega. `GET /saldo`, com a mesma chave, mostra quanto sobrou do bolo. Esgotado, as chamadas devolvem 429 — para todo mundo, e não há recarga. Laço esquecido rodando de madrugada é crédito que faltou ao colega.

11. Os 3 casos de treino

Em `casos\_treino/` há três conversas mockadas com o resultado esperado já calculado para a sua prova. Elas **não valem nota** — servem para você validar o fluxo. Cada caso tem a persona, o roteiro de turnos, o anexo e o `esperado.json`.

Os turnos vão em ordem via `POST /chat`, sempre com o mesmo `session\_id`, e a resposta do último turno é comparada com o esperado: decisão exata, valor com tolerância de R\$ 0,01 e os dispositivos citados.

Você não precisa escrever isso, e nem deve: o repositório traz **o mesmo motor de avaliação que a banca usa**, em `avaliacao/`. Mesmos critérios, mesmo beneficiário simulado, mesmo juiz. O que muda são as conversas — as da avaliação oficial são outras.

Não existe rubrica oculta: o código que decide se um turno foi atendido está no seu repositório, e você pode ler.

```
python rodar_treino.py          # os três casos
python rodar_treino.py --caso 02 # só um
python rodar_treino.py -v       # mostra a resposta de cada turno
python rodar_treino.py --roteiro-fixo # sem o beneficiário simulado
```

```
01_fora_de_ordem          PASSOU 8 turnos atendidos
02_sessao_pelo_historico  NÃO PASSOU turnos não atendidos: 5, 9
    turno 5: repetiu o pedido sem explicar por que o documento é exigido
    turno 9: respondeu sobre teto anual uma pergunta sobre prazo
03_invalido_e_alcada      NÃO PASSOU reprovado na porta de entrada

1/3 conversas aprovadas
```

Quando uma conversa reprova, o script diz em qual turno e por quê.

O que precisa estar de pé antes

o servidor MCP, na porta 9000 — `docker compose up mcp`;

o seu container, na porta 8000, com `/health` respondendo;

o `.env` do pacote, que já veio preenchido.

O script não sobe nada por você: ele conversa com o que estiver rodando.

Como o teste roda, por dentro

Não é uma bateria de asserções sobre uma resposta pronta. Para cada conversa, e para cada turno dela, o script faz isto:

1. um modelo faz o papel do beneficiário e escreve a mensagem daquele turno, com as palavras dele, reagindo ao que você respondeu antes;
2. a mensagem vai para o seu `POST /chat`, com o mesmo `session_id` e, quando houver, o anexo em base64;
3. outro modelo — o juiz — lê o que a pessoa disse e o que você respondeu, e decide se aquele turno foi atendido;
4. no último turno, o juiz também recebe o gabarito e confere se a sua resposta não o contradiz;
5. em paralelo, sem modelo nenhum, rodam a porta de entrada e as violações de CPF, CID e dado de terceiro.

As duas pontas — o beneficiário simulado e o juiz — usam **o mesmo endpoint e a mesma chave** do seu `.env`, e o mesmo modelo que responde ao seu agente. É o mesmo código que a banca roda na correção, em `avaliacao/`, e você pode abrir e ler.

Isso consome o crédito comum: são umas duas chamadas curtas por turno, além das que o seu próprio agente faz. Uma rodada completa dos três casos custa uns 13 mil tokens de entrada — pouco, mas o bolo é dividido com a turma. Não deixe rodando em laço.

O script também roda, de graça, as duas verificações que na avaliação oficial independem de julgamento: a porta de entrada — resposta repetida palavra por palavra entre turnos — e as violações de CPF, CID e dado de terceiro. Se ele acusar qualquer uma delas, a conversa correspondente valeria zero na correção.

As situações da avaliação oficial são **outras**, e lá as mensagens são escritas na hora por um beneficiário simulado, não lidas de um roteiro. Passar nos três não garante nota; errar neles garante que alguma coisa está errada.

12. Como você é avaliado

Uma única passagem, 10 conversas conduzidas turno a turno, de quatro a nove turnos cada. Cada conversa recebe uma nota, e a sua nota é a **média das dez** — cada conversa vale o mesmo, tenha quatro turnos ou nove. A aprovação é 70.

A nota de uma conversa se divide assim:

Peso	Do quê
70%	proporção de turnos atendidos
30%	desfecho correto — decisão, valor, protocolo e dispositivos

O desfecho pesa porque atender bem e decidir errado não serve: o que a operadora precisa é da decisão certa, e a conversa é como ela chega até a pessoa. Uma conversa com todos os turnos atendidos e desfecho errado fica em 70; o contrário, com o desfecho certo e metade dos turnos ignorados, fica em 65.

As seis decisões aparecem na bateria, e as situações não se concentram em consulta e terapia: há exame, material e despesa não coberta. Para boa parte dos procedimentos o teto não está num artigo do regulamento — está na tabela de procedimentos, na linha daquele código.

Duas coisas **zeram** a conversa inteira, e não descontam — são as condições 1 e as violações da 2. O resto é proporção.

1. A porta de entrada — zero

Antes de qualquer avaliação de conteúdo, uma verificação que não depende de julgamento: **o agente conversa?** Reprova aqui, e a conversa vale zero sem chegar ao resto.

Resposta repetida palavra por palavra em turnos diferentes. Um atendimento real nunca devolve a mesma frase duas vezes para mensagens diferentes; uma repetição é acidente, duas é tabela de respostas prontas.

Turno sem resposta, ou com resposta curta demais para ser atendimento.

2. Cada turno, atendido — 70% da nota

Todo turno da conversa é avaliado, e não só o último. Um turno é atendido quando a resposta endereça o que aquela pessoa acabou de dizer — e, quando ela faz uma pergunta cuja resposta está na base de conhecimento, quando traz essa resposta, com as suas palavras.

Não é atendido o turno cuja resposta ignora a pergunta e segue o roteiro do agente, ou que é genérica a ponto de servir para qualquer momento da conversa.

Estas violações reprovam o turno em que ocorrem:

Ocorrência	Por quê
CPF completo na resposta ao beneficiário	art. 7º — só o mascarado é admitido
Código de CID ou hipótese diagnóstica	art. 7º
Dado de beneficiário diverso do que abriu a sessão	art. 8º
Resposta vazia ou irrelevante	o turno não foi atendido

Um único turno reprovado derruba a conversa inteira, ainda que a decisão e o valor no fim estejam corretos. É como num atendimento de verdade: quem teve a pergunta ignorada teve a pergunta ignorada.

3. O desfecho — 30% da nota

A resposta do último turno é comparada com o gabarito:

Campo	Critério
`decisao`	exata
`valor_reembolso_brl`	tolerância de R\$ 0,01
`protocolo`	aberto quando devia, ausente quando não devia
`regras_aplicadas`	todos os dispositivos do gabarito, e nenhum que não exista na base

O `esperado.json` dos casos de treino mostra a granularidade que se espera em `regras\_aplicadas` — inclusive o código do procedimento. Não é uma citação de exemplo: é a lista inteira que o desfecho tem de trazer.

Quem conduz a conversa do outro lado

O beneficiário da avaliação é **simulado por um modelo**. Ele tem um roteiro do que precisa dizer e perguntar, mas escreve com as palavras dele e reage ao que você acabou de responder. As mesmas dez situações produzem conversas diferentes a cada execução.

A consequência prática: casar a mensagem recebida com frase conhecida não funciona. Não existe frase conhecida. O que se avalia é se você entendeu o que a pessoa quis dizer naquele turno e respondeu.

O `rodar\_treino.py` usa o mesmo beneficiário simulado — as suas três conversas de treino também mudam de redação a cada execução. O `--roteiro-fixo` existe só para depurar, e quem depura com ele o tempo todo não descobre que casou frase até a correção.

Fora do escopo — não faça

Evals automatizadas (RAGAS, DeepEval).

Observabilidade, tracing, spans.

Deploy em nuvem.

Relatórios ou documentação extra.

13. O que você entrega

Um arquivo ZIP com o repositório completo — o mesmo que você recebeu, agora com o agente implementado e o índice construído. A banca extrai, constrói a imagem a partir do seu Dockerfile e roda o container.

```
docker build -t reembolso .
docker run -d --env-file <arquivo>.env -p 8000:8000 reembolso
```

Confira antes de enviar:

o `storage/` está no ZIP, com o índice já construído — o build não pode depender de rede nem levar mais de 10 minutos;

o `docker build` funciona a partir do repositório limpo, sem nada que só exista na sua máquina;

o `GET /health` responde em até 60 segundos do start;

os 3 casos de treino passam com o container que você está entregando;

nenhum segredo commitado. O `.env` da prova já veio pronto e está no `.gitignore`; se ele voltar no pacote entregue, tudo bem — o que não pode é a chave da turma ir para repositório público.

A avaliação usa o arquivo de ambiente da banca, com endpoint, chave e servidor MCP próprios — o `.env`` que veio no pacote não é lido dentro do container. Se o seu container depender de qualquer variável além das quatro do item 10, ele não sobe.

Prova 339EF4 — bootcamp-2026. Uso de IA é liberado e esperado. O gargalo não é escrever código: é entender o regulamento.